What is indexing?

→ helps – if data retrieval speed is slow

- Method that helps us to locate a record or data present in the memory faster
- → Boosts efficiency by minimizing the number of disk accesses required when we process a query.
- → Speeds up operations with READ
  - → SELECT Queries
  - → WHERE Clause

Indexing Methods

- Primary Index
  - → Index has
    - → Search → Primary key
    - → Data Reference
  - Dense Index
    - ↓
    - Data is sorted based on the primary key.
  - Sparse Index
    - not every primary key has index
    - → space efficiency → better
    - → speed → ↓ as compared to dense
- Clustering Index
- Secondary Index

Each and every primary/ search key will have an index.

Dense Index → data structure
→ not space friendly
→ retrieval speed – better

- Sparse Indices can only be used if relation is stored in sorted order of the search key

A block can hold either 6 records or 12 key pointers. A database contains 48 records, then how many blocks is required to hold the data file and the dense index?

Solution description

Ans: 12 Explanation: We know, the database contains 48 records and a block can hold 6 records or hold 12 key pointers. Therefore, how many blocks can hold 48 records? 48/6 = 8 Similarly, how many blocks for 12 key pointers? 48/12 = 4 Hence, total blocks = 8+4=12.

<u>Clustering index</u>
→ Ordered Data files
→ Search < Not primary Key
Not even → unique identifier
→ part of candidate key
→ Data Reference → K Pointer
→ Collection of other attributes which can uniquely identify tuple.

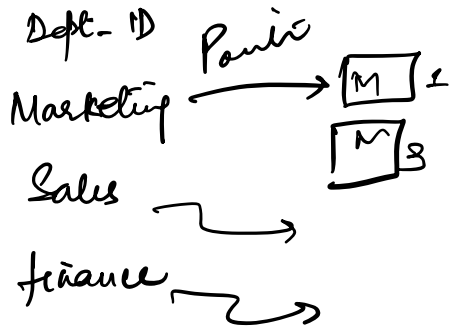
- Used? → Sometimes we want to group data according to something & then search it.

Emp Table

Emp_ID	Dept_ID	Salary
1	Marketing	$1000
2	Sales	$1200
3	Marketing	$1100
4	finance	$1300

Cluster on this

(Add index here)



Secondary Index

Primary Index < Dense / Sparse

↓

sits on RAM

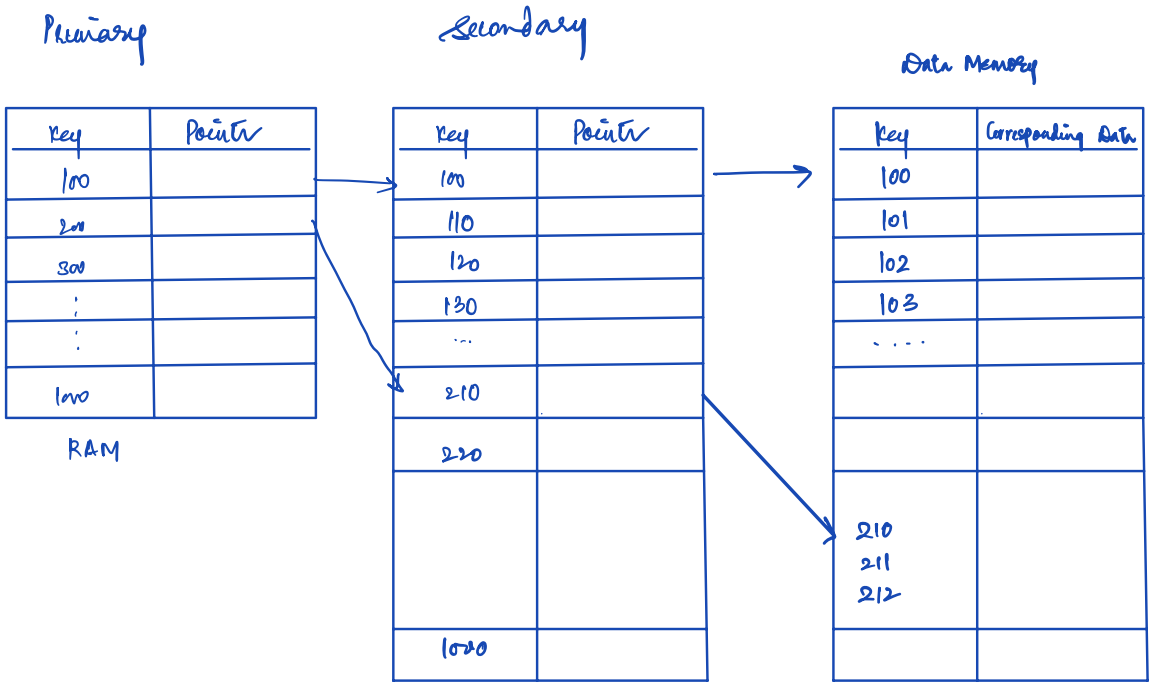
What if primary key goes till lakhs?

Even sparse won't make difference.

So we use secondary index.

↓

Hard Disk



How to fetch data for 212? → find the value just smaller than 212 in primary index → 200 → It will find value nearest to 200 → 210 → It will see if second index after this is bigger than 212 or smaller than 212 → 210 → then go to memory and search for exact index.

Drawbacks

→ Space constraints

→ Insert / Update / delete the data → Operations become heavy → lots of indexes.